Student Record Management System

Description

This is a **Student Record Management System developed in C** using a **Singly Linked List (SLL)**.

The program stores student details such as:

- Roll Number
- Name
- Percentage
- Next node address

The program provides a menu-driven interface to perform different operations on student records.

Data Structure

The program uses a **Singly Linked List**.

Each node is represented using the following structure:

```
struct student
{
    int rollno;
    char name[50];
    float percentage;
    struct student *next;
};
```

`SLL` is used as the typedef name for this structure.

Menu Options

The program provides the following options:

```
a/A : Add new record
d/D : Delete a record
s/S : Show the list
m/M : Modify a record
v/V : Save records
e/E : Exit
t/T : Sort the list
l/L : Delete all the records
r/R : Reverse the list
```

Features

1. Add New Record

Option:

```
a/A
```

Adds a new student record to the linked list.

The program automatically assigns the **smallest available roll number**, starting from `1`.

The user enters:

- Student name
- Percentage

Example:

```
Enter student name: Rahul
```

```
Enter percentage: 85.5
```

2. Delete a Record

Option:

```
d/D
```

A student record can be deleted using either:

```
R/r : Enter roll number to delete
N/n : Enter name to delete
```

Delete by Roll Number

The program searches for the specified roll number and removes the corresponding node.

Delete by Name

The program searches for the specified name.

If more than one record has the same name, the matching records are displayed.

3. Show the List

Option:

```
s/S
```

Displays all student records currently present in the linked list.

Example:

```
Roll No.  Name    percentage
1 Rahul   85.50
2 Ravi    78.25
3 Kumar   91.00
```

4. Modify a Record

Option:

```
m/M
```

A record can be searched using:

```
R/r : Search by roll number
N/n : Search by name
P/p : Search by percentage
```

After finding the record, the program allows the user to update:

- Name
- Percentage

If multiple records have the same name or percentage, the program asks for the roll number to identify the required record.

5. Save Records

Option:

```
v/V
```

Saves the linked-list records into:

```
student.dat
```

The data is written using:

```
rollno name percentage
```

6. Sort the List

Option:

```
t/T
```

The program provides two sorting options:

```
N/n : Sort with name
P/p : Sort with percentage
```

Sort by Name

Records are arranged alphabetically according to the student name.

Sort by Percentage

Records are arranged according to percentage in **descending order**.

The sorting is performed by exchanging the data between nodes rather than changing the node links.

7. Delete All Records

Option:

```
l/L
```

Deletes every node from the linked list.

The dynamically allocated memory for every node is released using `free()`.

8. Reverse the List

Option:

```
r/R
```

Reverses the order of the student records.

The function:

```
reverseallrecord(SLL **ptr)
```

uses dynamically allocated memory for an array of `SLL *` pointers and rearranges the `next` links to reverse the list.

9. Count Nodes

The function:

```
int countNode(SLL *ptr)
```

counts the total number of nodes present in the linked list.

This function is also used by the sorting and reverse operations.

Functions Used

`add_record()` – Adds a new student record

`deleterecord()` – Provides delete options

`delete_rollno()` – Deletes a record using roll number

`delete_name()` – Deletes a record using name

`showrecord()` – Displays all records

`modifyrecord()` – Provides modification options

`modify_rollno()` – Modifies a record searched by roll number

`modify_name()` – Modifies a record searched by name

`modify_percentage()` – Modifies a record searched by percentage

`saverecord()` – Saves records into student.dat

`sortrecord()` – Provides sorting options

`accord_to_sort_name()` – Sorts records by name

`accord_to_sort_percentage()` – Sorts records by percentage

`deleteallrecord()` – Deletes all nodes

`reverseallrecord()` – Reverses the linked list

`countNode()` – Counts linked-list nodes

Concepts Used

This project demonstrates the following C programming and data-structure concepts:

- Structures
- `typedef`
- Singly Linked List
- Pointers
- Double Pointers
- Dynamic Memory Allocation
- `malloc()`
- `free()`
- Functions
- String handling
- `strcmp()`
- `strcpy()`
- File handling
- `fopen()`
- `fprintf()`
- Searching
- Sorting
- Linked-list traversal
- Node insertion
- Node deletion
- Linked-list reversal

File Handling

The program saves student records into:

```
student.dat
```

The file is opened in write mode:

```
fp = fopen("student.dat", "w");
```

Records are written using:

```
fprintf(fp, "%d %s %f\n",
        ptr->rollno,
        ptr->name,
        ptr->percentage);
```

Dynamic Memory Allocation

A new student node is dynamically allocated using:

```
new = malloc(sizeof(struct student));
```

When a record is deleted, its memory is released using:

```
free(del);
```

The delete-all operation releases the memory of every node.

Compilation

If the program is divided into multiple `.c` files, compile all source files together.

Example:

```
gcc stud_main.c stud_add.c stud_del.c stud_show.c stud_mod.c stud_save.c stud_sort.c stud_deleteall.c stud_re
```

Run the program:

```
./student
```

The header file should be included in the source files:

```
#include "heade.h"
```

The header file should **not** be compiled separately.

Example Menu

```
*********** STUDENT RECORD MENU ************
a/A : Add new record
d/D : Delete a record
s/S : Show the list
m/M : Modify a record
v/V : Save records
e/E : Exit
t/T : sort the list
l/L : Delete all the records
r/R : Reverse the list

Enter your choice :
```

Project Objective

The main objective of this project is to implement a **menu-driven student record management system using a singly linked list** and to practice dynamic memory allocation, pointers, searching, sorting, file handling, and linked-list operations in C.

Author

Student Record Management System

Developed as a C programming and Data Structures project.